SPÉCIFICATION D'ARCHITECTURE

The Bill Please : de la facture déposée à la ligne dans le tableau

Plateforme web multi-clients qui lit des factures fournisseurs (PDF, photos, archives .zip ; imprimées ou manuscrites ; français ou anglais) avec l'API Claude, puis les présente dans un tableau de bord et un tableau corrigeable. Ce document décrit le trajet d'une facture, l'architecture qui le porte, la place exacte de Claude, et les temps mesurés.

État du code 17 septembre 2026 **Dépôt** BenoitPierson/The-bill-please

Mesures 24 factures réelles, 8 traitements en parallèle

SOMMAIRE

1. Pipeline d'une facture
   2.1 Vue d'ensemble
   2.3 Modèle de données
   2.5 Boucle d'apprentissage
2. Architecture
   2.2 Isolation des clients
   2.4 Zoom : Claude
3. Temps d'intégration
4. Temps restant
5. Limites et suites

≈ 10 s	71 %	1,6 s	≈ 31 min	39
pour traiter une facture seule (médiane)	de ce temps passé dans l'appel à Claude	par facture en lot, 8 en parallèle	pour le lot de référence de 1 181 factures	doublons détectés sur 1 181, soit 100 Mo évités

Temps mesurés les 16 et 17 septembre 2026 sur le jeu « Factures Dijo » ; ils dépendent de la latence de l'API au moment du traitement.

1 Pipeline d'une facture

L'import est découpé en deux temps. Le navigateur envoie les fichiers et reçoit aussitôt un numéro de lot : il n'attend pas l'extraction. Le serveur traite ensuite chaque document dans un fil d'exécution en arrière-plan, huit à la fois, pendant que la page interroge l'avancement toutes les deux secondes.

Chaque document passe un contrôle de doublon *avant* tout stockage ou appel à Claude : une copie exacte ne coûte donc ni espace, ni appel facturé, et ne gonfle pas les totaux.

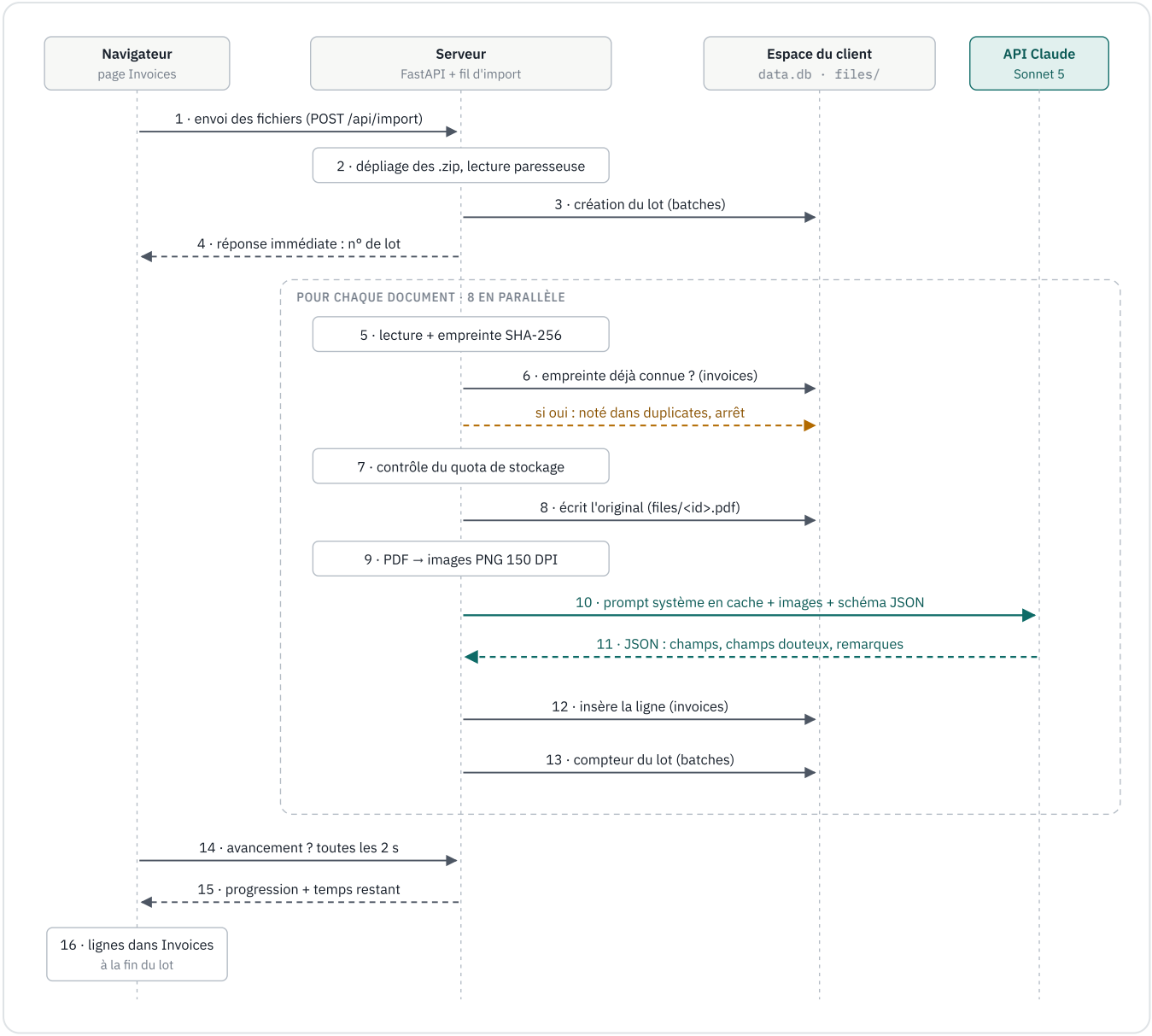


Figure 1. Séquence complète. En sarcelle, le seul échange avec Claude (étapes 10 et 11) ; en ambre pointillé, la sortie anticipée d'un doublon, qui n'atteint jamais le stockage ni Claude. Les étapes 5 à 13 se répètent pour chaque document, huit documents à la fois.

ÉTAPES	CE QUI SE PASSE	CODE	TEMPS MÉDIAN
1–4	Envoi des fichiers, dépliage des archives, création du lot. Le navigateur reçoit le numéro de lot sans attendre l'extraction.	`app.py · pipeline.start_import`	instantané*
5	Lecture du document depuis l'archive, puis empreinte SHA-256 de ses octets.	`_process_one`	0,03 s
6	Recherche de l'empreinte dans le lot en cours puis dans la base. Copie exacte : journalisée et abandonnée.	`db.find_by_fingerprint`	0,05 s
7	Vérification que le document tient dans le quota du client.	`_BatchState`	< 0,01 s
8	Écriture du document original, consultable plus tard depuis le tableau.	`_store_original`	0,01 s

ÉTAPES	CE QUI SE PASSE	CODE	TEMPS MÉDIAN
9	Rendu de chaque page (5 au maximum) en image PNG à 150 DPI.	`file_to_page_images`	1,2 s
10–11	Appel à Claude : lecture des images, réponse JSON contrainte par un schéma.	`extract_invoice`	6,9 s
12–13	Insertion de la ligne et incrément du compteur du lot.	`db.insert_invoice`	0,06 s
14–16	La page suit l'avancement, affiche le temps restant, puis se recharge à la fin du lot.	`static/app.js`	toutes les 2 s

* Hors durée d'envoi, qui dépend de la connexion : environ 0,5 s pour une facture de 1,1 Mo à 20 Mbit/s, 2 minutes pour une archive de 300 Mo.

Les chemins de sortie

Un document ne produit pas toujours une ligne extraite. Chaque cas reste visible dans l'outil : rien ne disparaît sans explication.

SITUATION	RÉSULTAT	APPEL À CLAUDE
Copie exacte d'une facture déjà importée	Ignorée, listée dans « Doublons ignorés » avec un lien vers l'original	Non
Copie d'une facture dont l'extraction avait échoué	Retraitée ; la ligne en erreur est remplacée	Oui
Copie d'une ligne en erreur corrigée à la main	Considérée comme doublon : le travail manuel est conservé	Non
Quota de stockage atteint	Ligne en erreur « Quota de stockage atteint »	Non
Fichier ni PDF ni image (Word, Excel…)	Ligne en erreur « Format non pris en charge »	Non
API surchargée (erreur 429 ou 529)	Jusqu'à 6 nouvelles tentatives espacées, puis ligne en erreur	Oui, répété

2 Architecture

Un seul processus Python (FastAPI) sert toutes les entreprises clientes. Il n'y a ni file de messages, ni service séparé : l'import tourne dans un fil d'exécution du même processus. Ce choix garde la plateforme simple à déployer et à maintenir tant que le volume reste de l'ordre de quelques milliers de factures par client ; ses limites sont listées en section 5.

2.1 Vue d'ensemble

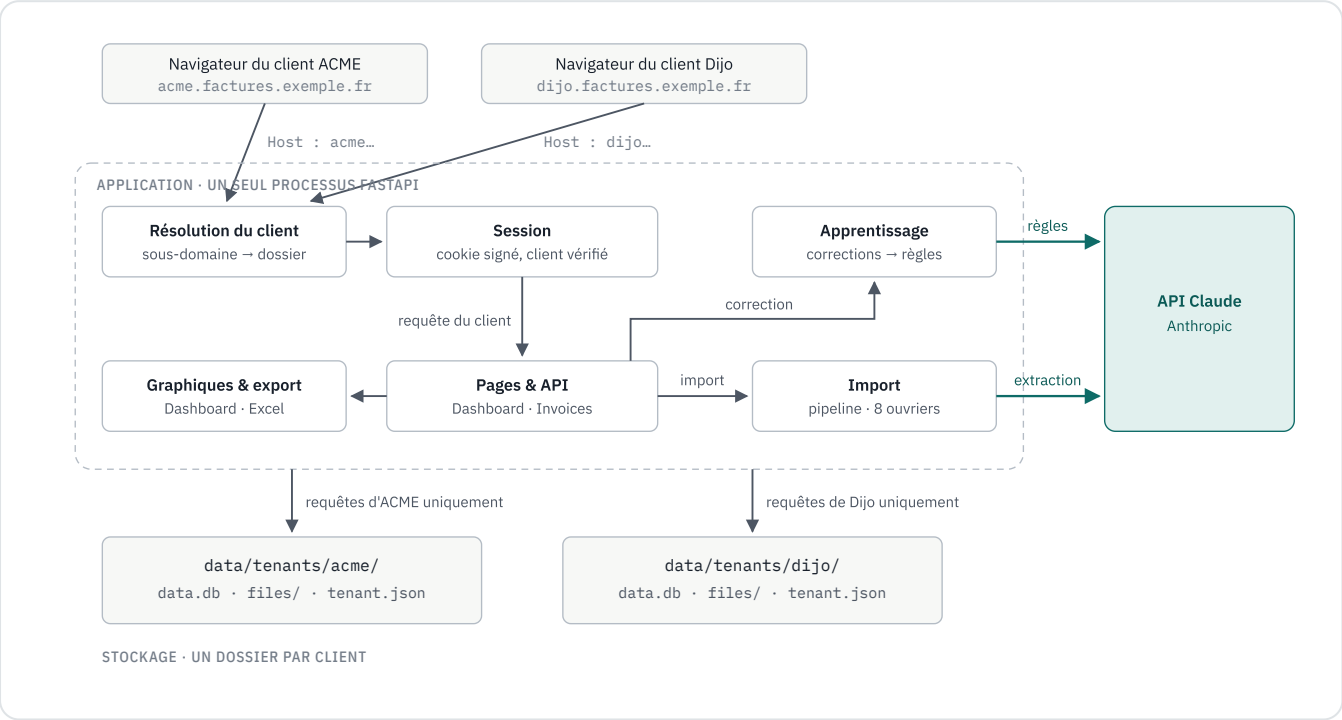


Figure 2. Une application, des espaces cloisonnés. Le sous-domaine désigne le client, la session doit appartenir à ce même client, et toute lecture ou écriture se fait dans le dossier de ce client. Claude est sollicité par deux modules seulement : l'import (une requête par facture) et l'apprentissage (une requête par fournisseur corrigé).

MODULE	RÔLE
`app.py`	Routes web et API, pages, session, export Excel à la demande.
`tenants.py`	Résolution du client depuis le sous-domaine ; le système de fichiers sert de registre. Mesure de l'espace et quota.
`auth.py`	Mots de passe hachés en scrypt (bibliothèque standard, rien à compiler).
`db.py`	Schéma, migrations additives et requêtes, par client.
`pipeline.py`	Import en arrière-plan : empreinte, doublons, quota, stockage, extraction parallèle, estimation du temps restant.
`extraction.py`	Dépliage des archives, conversion en images, prompt et appel à Claude.
`learning.py`	Transformation des corrections en règles d'extraction.
`charts.py · excel_export.py`	Géométrie des graphiques du Dashboard ; export Excel coloré comme le tableau.
`manage.py`	Administration : création des espaces et comptes, espace occupé, quotas.

2.2 Isolation des clients

La décision structurante de l'architecture. Deux façons de loger plusieurs clients étaient possibles ; la figure montre ce qui les distingue.

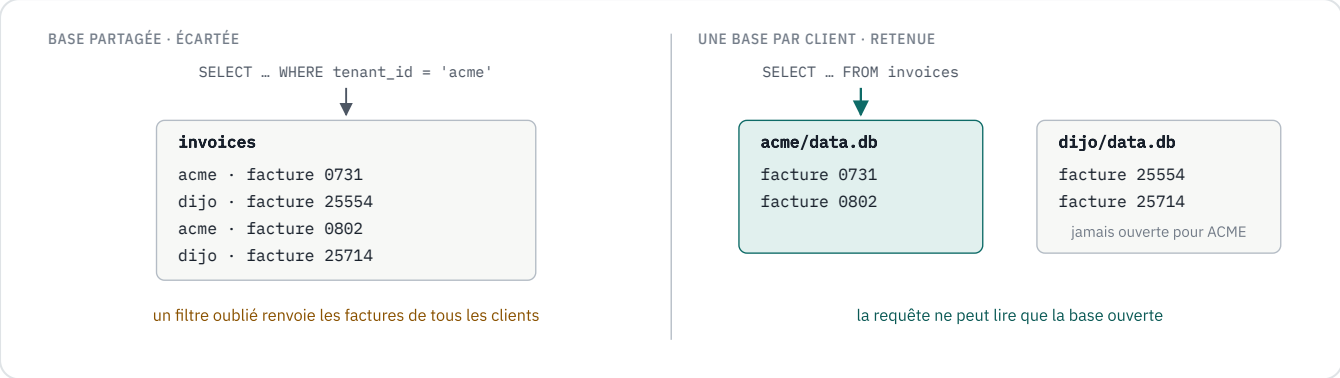


Figure 3. Avec une base partagée, la sécurité repose sur un `WHERE` présent dans chaque requête, écrite par chaque développeur, pour toujours. Avec une base par client, il n'y a pas de filtre à oublier. Contrepartie : un changement de schéma doit s'appliquer à chaque base, ce que `db.py` fait automatiquement à la première ouverture.

Deux verrous complètent cette isolation :

- **La session porte le client.** Un cookie émis pour `acme` est refusé sur `dijo`, même si le mot de passe est identique.
- **Les originaux ne sont jamais servis comme fichiers statiques.** Chaque consultation vérifie la session et recalcule le chemin à l'intérieur du dossier du client, de sorte qu'un nom de fichier forgé ne peut pas en sortir.

Bénéfice direct pour le RGPD : sauvegarder, exporter ou supprimer les données d'un client revient à traiter un dossier.

2.3 Modèle de données

Six tables, identiques dans la base de chaque client.

TABLE	CONTENU	COLONNES CLÉS
`users`	Comptes de l'espace client	`email, password_hash`
`batches`	Un lot par import, et son avancement	`total, done, failed, duplicates, status`
`invoices`	Une ligne par facture : données extraites, doutes, corrections, trace du document	`supplier_name … vat_amount, uncertain_fields, edited_fields, content_sha256, stored_name`
`duplicates`	Documents ignorés car identiques à une facture existante	`filename, duplicate_of, file_size`
`corrections`	Journal brut des modifications faites à la main	`field, old_value, new_value, learned`
`learned_rules`	Consignes déduites des corrections, une par fournisseur	`supplier_name, rule, based_on`

2.4 Zoom : l'intégration de Claude

Chaque facture donne lieu à **un seul appel**, qui transporte toutes ses pages ensemble : le total et la TVA d'une facture de plusieurs pages figurent souvent dans le récapitulatif final, qu'il faut voir en même temps que l'en-tête. La réponse n'est pas du texte libre à interpréter : un schéma JSON impose la forme exacte, champ par champ.

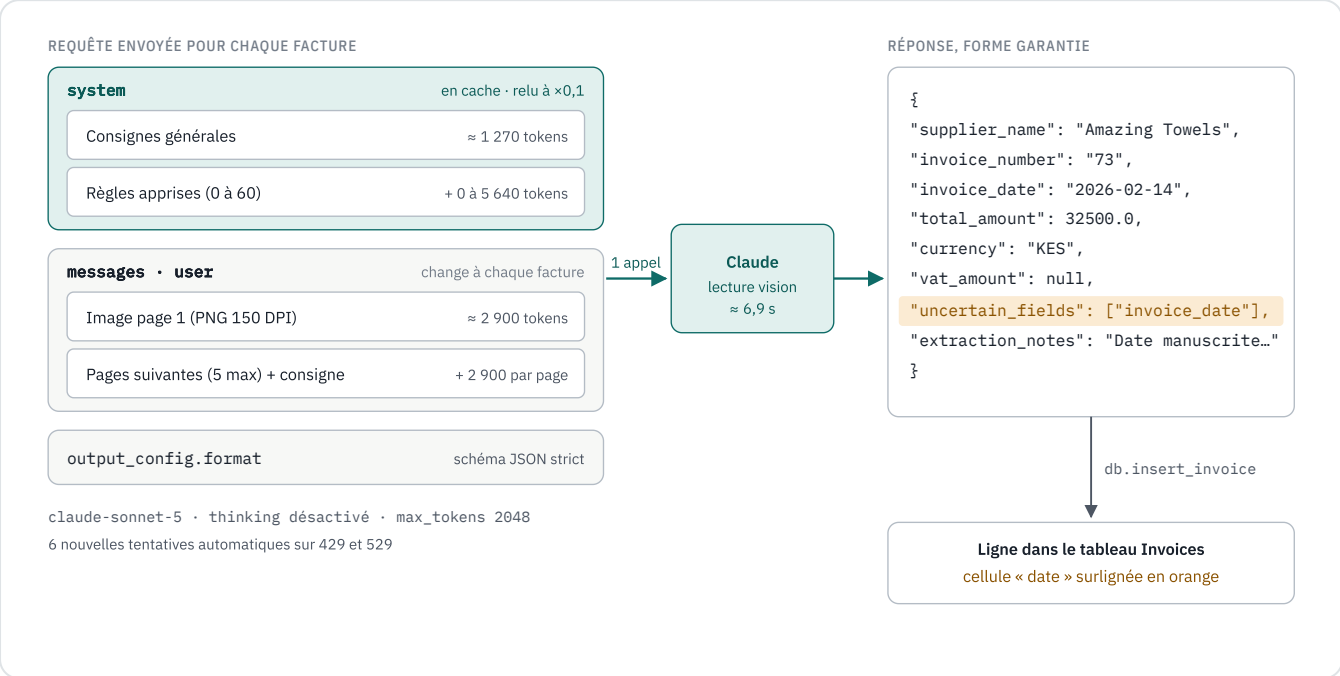


Figure 4. Ce qui part, ce qui revient. La partie système (sarcelle) est identique pour toutes les factures d'un lot : elle est mise en cache et facturée au dixième après le premier appel. Les images changent à chaque facture. Le champ `uncertain_fields` (ambre) est une liste structurée, pas une phrase : c'est ce qui permet à l'interface de surligner précisément les cellules douteuses.

Les choix qui encadrent cet appel :

- **Normalisations demandées dans le prompt, pas en Python.** Dates toujours au format `AAAA-MM-JJ`, devises en code ISO 4217 (« Ksh », « SHS » ou « Kshs » deviennent `KES`), montants en nombres. Le numéro de facture ne doit jamais être le *Receipt Number* ni le long numéro fiscal imprimé près du code-barres.
- **Réflexion désactivée.** Il s'agit de lire, pas de raisonner : la désactiver rend le temps et le coût prévisibles.
- **Mise en cache du prompt système**, mesurée sur appels réels : environ 2 500 tokens écrits au premier appel puis relus au dixième du prix, soit environ 7 $ économisés sur un lot de 1 181 factures avec 8 règles apprises. Sans cache, 60 règles doubleraient le coût par facture (6 910 tokens de prompt contre 2 900 pour l'image).
- **Sortie structurée transmise par `extra_body`.** La version installée du SDK Python (0.72.0) ne connaît pas encore le paramètre `output_config` ; il est donc passé tel quel dans le corps de la requête, que l'API accepte.
- **Nouvelles tentatives** portées à 6 : les 2 tentatives par défaut ne suffisaient pas lors des pics de surcharge de l'API (erreur 529).

> **Données transmises.** Les images des factures quittent le serveur pour l'API d'Anthropic, hébergée hors de l'Union européenne. En production, ce transfert doit être encadré contractuellement (accord de traitement des données) et signalé aux clients comme sous-traitance.

2.5 Boucle d'apprentissage des corrections

Claude intervient une seconde fois, hors import. Une correction brute (« 0190485190000003873 » remplacé par « BC-2026-0211 ») est une valeur : la rejouer n'apprendrait rien pour la facture suivante. Ce qui se généralise, c'est la raison de la correction. Un appel distinct, sans image, la formule en une consigne.

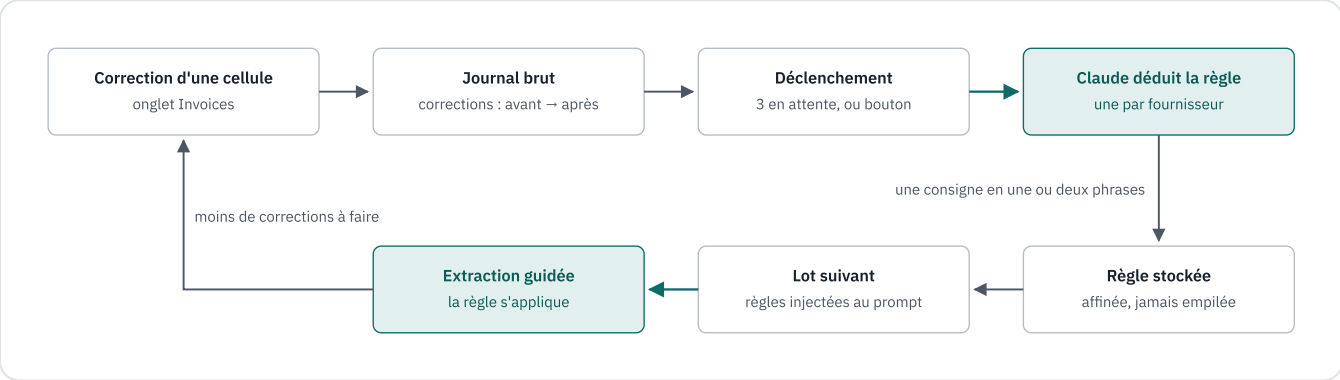


Figure 5. Les corrections restent conservées telles quelles (la vérité de référence, toujours auditable) ; seules les règles qui en sont tirées entrent dans le prompt. Une règle est reformulée à chaque nouvelle série de corrections plutôt qu'ajoutée à la suite, ce qui garde le prompt court quelle que soit la durée d'utilisation.

Règle effectivement produite par Claude après trois corrections sur les factures de test :

> Pour ce fournisseur, ne pas utiliser le long numéro figurant près du code-barres comme numéro de facture ; rechercher plutôt la référence au format « MAF-XXXXX » et retenir celle-ci comme numéro de facture.
>
> learned_rules · Majid Al Futtaim Hypermarkets (Carrefour)

Toutes les règles du client sont envoyées à chaque facture, car le fournisseur n'est connu qu'*après* l'extraction. Si leur nombre devenait trop grand malgré le cache, le nom du dossier dans l'archive (`Paid Feb/Carrefour/…`) permettrait de ne sélectionner que les règles pertinentes avant l'appel.

3 Temps d'intégration d'une facture

Chronométrage de chaque étape sur 24 factures réelles choisies pour couvrir toutes les tailles de fichier (de 0,1 à 16 Mo), traitées comme en production : 8 à la fois, cache du prompt actif, aucune erreur.

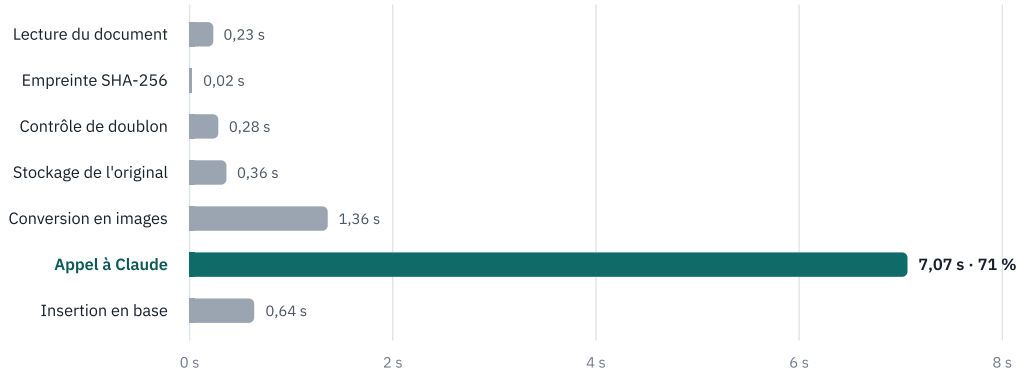


Figure 6. Temps moyen par étape, pour une facture (total moyen : 9,96 s). Les moyennes sont utilisées ici pour que les étapes s'additionnent ; les médianes figurent au tableau de la section 1.

SCÉNARIO	TEMPS MESURÉ OU PROJETÉ	REMARQUE
Une facture seule, du dépôt à la ligne	≈ 10 s	médiane 9,8 s ; maximum observé 16,8 s
Facture d'une page / de 4 pages et plus	6,9 s / 10,7 s	temps de l'appel à Claude seul
Fichier scanné de plus de 3 Mo	+ 3 s	conversion en images plus lourde
Lot de 24 factures	37,8 s	mesuré, 1,58 s par facture
Lot de référence : 1 181 factures	≈ 31 min	projection à 1,58 s par facture ; moins avec des doublons

Trois enseignements :

- **Claude fixe le rythme.** Avec 71 % du temps, seul le parallélisme accélère réellement un lot. Aller au-delà de 8 traitements simultanés dépend des limites de débit du compte Anthropic.
- **Le parallélisme n'est pas gratuit.** Huit traitements totalement indépendants prendraient 30 s pour 24 factures ; on en mesure 37,8. L'écart vient surtout des écritures concurrentes dans SQLite : l'insertion, 0,06 s en médiane, monte jusqu'à 3 s quand plusieurs traitements écrivent en même temps.
- **La latence de l'API varie.** Les mêmes factures peuvent prendre sensiblement plus longtemps lors d'une période de charge : ces chiffres sont des ordres de grandeur, pas des garanties.

4 Temps restant affiché pendant l'import

Pendant un import, la page affiche sous la barre de progression le temps restant estimé et le temps écoulé, par exemple « Temps restant estimé : ~28 min · écoulé : 3 min 45 s ». Le compte à rebours diminue chaque seconde et se recale à chaque réponse du serveur, toutes les deux secondes.

L'estimation est calculée côté serveur, en deux régimes :

```
Avant les premiers résultats (« première estimation ») :
  restant = ⌈ factures à extraire ÷ 8 ⌉ × 10 s × 1,25

Dès qu'une première série de 8 extractions est terminée (« mesurée ») :
  restant = factures à extraire × (temps écoulé ÷ extractions terminées)

factures à extraire = factures restantes × (1 − part de doublons observée)
```

- **10 s** est le temps médian mesuré par facture, et **1,25** le surcoût de parallélisme observé (37,8 s contre 30 s). Ensemble, ils prédisent 31 minutes pour 1 181 factures, en accord avec la mesure.
- **Le régime mesuré s'adapte à la réalité du moment** : latence de l'API, nombre de pages des factures de ce lot, charge de la machine.
- **Les doublons sont comptés à part**, car ils se traitent en quelques millisecondes : les inclure ferait croire à un import presque terminé.
- **En fin de lot**, les dernières factures tournent en parallèle ; l'estimation garde au moins une demi-extraction tant qu'il reste du travail, pour ne pas afficher « quelques secondes » pendant 7 s.

Testé sur un lot simulé de 72 documents dont 12 doublons : une fois en régime mesuré, l'écart médian entre temps estimé et temps réel est de **9 %**.

> **Import interrompu.** L'import vit dans le processus du serveur : un redémarrage l'arrête. La page le détecte (lot marqué « en cours » sans traitement actif), l'indique au lieu d'afficher un compte à rebours figé, et invite à redéposer les fichiers. Grâce au contrôle des doublons, les factures déjà intégrées sont alors reconnues et ignorées : seul le reste est traité.

5 Limites connues et suites

LIMITE	CONSÉQUENCE	ÉVOLUTION ENVISAGÉE
Imports dans le processus web	Un redémarrage interrompt l'import en cours ; un seul serveur possible	File de tâches (worker séparé) si plusieurs serveurs deviennent nécessaires
Écritures SQLite concurrentes	Principale cause des 20 % de temps de lot perdus en contention (37,8 s mesurées contre 30 s théoriques)	Un fil d'écriture unique qui regroupe les insertions
Originaux sur le disque du serveur	≈ 2 Go par client et par an ; sauvegardes à gérer	Stockage objet compatible S3, archivage à froid après quelques mois
Un document = une facture	Un PDF regroupant plusieurs factures n'en restitue qu'une, avec une remarque	Détection et découpage des factures multiples
Traitement synchrone de l'API	Prix plein	API Batches : −50 % sur le coût, pour des lots qui peuvent attendre jusqu'à 24 h
Mise en production	Fonctionne aujourd'hui en local	HTTPS, DNS joker `*.domaine`, variables `APP_DOMAIN` et `SESSION_SECRET`, hébergement en UE

Document établi à partir du code du dépôt et de mesures effectuées les 16 et 17 septembre 2026 sur les archives « Factures Dijo » (1 181 documents). Les temps et coûts dépendent de la latence et de la tarification de l'API Claude au moment du traitement.